# Synthesising the uNPU from scratch

From SystemVerilog to 12 487 SCL 180 nm cells: libraries, timing, constraints, Design Compiler, and how to read every report

EE5239 · Project 2 · Frankenstein 180 nm IP-Validation SoC
Matches `scripts/synth.tcl`, `syn/dc_setup.tcl`, `syn/dc_run.tcl`, `constraints/pm_unpu_top.sdc` and the reports of 15 Oct 2026 (`syn/out/`)
November 2026

**What is inside**


1. Where synthesis sits in the flow
2. Standard cells and the library
3. What Design Compiler makes of our RTL
4. Static timing in one chapter
5. The uNPU’s constraints
6. Running Design Compiler
7. Reading the timing report
8. Area, cells, power, rule checks
9. Three runs and one constraint lesson
10. Findings and recommendations
11. Sign-off checklist and hand-off
12. Command reference card

# 1. Where synthesis sits in the flow

| Step | Tool | Status for the uNPU |
|---|---|---|
| RTL simulation | Xcelium 22.09, Verilator 5.053 | done, all testbenches pass |
| **Logic synthesis** | **Design Compiler T-2022.03-SP5** | **done** (this document) |
| Pre-layout gate-level simulation | Xcelium | done, 343 341 checks pass |
| Floorplan, place, CTS, route | ICC2 U-2022.12-SP3 | in progress |
| DRC / LVS | Calibre | to do |
| Timing sign-off | PrimeTime | to do |

RTL describes *behaviour*: `psum_out <= psum_in + product`. A chip is built from *cells*: a 32-bit adder made of full-adder cells, 32 flip-flops, and the wires between them. **Synthesis** is the translation between the two. It must preserve behaviour exactly while meeting timing, and it should keep area and power small.

# 2. Standard cells and the library

A **standard cell** is a pre-designed, pre-characterised layout of one small logic function: an inverter, a NAND, a flip-flop. Every cell in a library has the same height, so cells can be placed side by side in rows. The SCL 180 nm FS120 library has 534 of them.

Names encode the function and the drive strength. For example:

| Cell | Function | Note |
|---|---|---|
| `inv0d1`, `inv0d2`, `inv0da` | inverter | `d1` weakest; higher numbers and letters are stronger |
| `buffd1`, `buffda` | buffer | `buffd1` is the input driver assumed in the SDC |
| `nr03d1` | 3-input NOR | |
| `an02d2` | 2-input AND | |
| `oai222d1` | OR-AND-INVERT, three 2-input ORs | typical in a read mux |
| `dfcrq1`, `dfcrn1` | D flip-flop with asynchronous clear | what an `always_ff ... or negedge rst_n` becomes |

A **Liberty file** (`.lib`, compiled to `.db` for DC) describes, for every cell, its area, its pins, and tables of delay and output slew as functions of input slew and output load. Those tables are measured at one **corner**:

| Library | Process | Temperature | Voltage | Used for |
|---|---|---|---|---|
| `tsl18fs120_scl_ss` | slow (1.2) | 125 °C | 1.62 V | **setup**: the slowest the chip will ever be (DC uses this) |
| `tsl18fs120_scl_ff` | fast (0.8) | −40 °C | 1.98 V | **hold**: the fastest (PrimeTime) |

There is no typical library in this kit. Units are 1 ns and 1 pF.

**Don't-use cells.** The kit's `doc/std_cell_guidelines.pdf` forbids six families: `slnht*`, `skbrb*`, `slbhb*`, `slnhn*`, `slnln*` and `mx08d*`. The script marks them `set_dont_use`, so DC never picks them.

**DesignWare** (`dw_foundation.sldb`) is Synopsys's library of arithmetic building blocks. When DC sees `*` or `+` in the RTL, it picks a DesignWare multiplier or adder architecture and then maps that onto cells.

# 3. What Design Compiler makes of our RTL

Every RTL construct has a predictable gate-level result, and the reports show it.

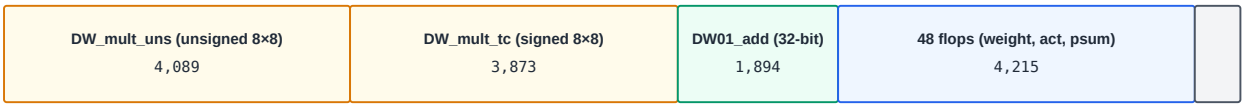
| RTL | Becomes | Seen in |
|---|---|---|
| `always_ff @(posedge clk or negedge rst_n)` | async-clear flip-flops (`dfcrq1`, `dfcrn1`) | `timing_hold.rpt` start/end points |
| `psum_in + product` (32 bit) | `DW01_add` | `area.rpt`: `.../pe/add_42`, 1 894 µm² each |
| `$signed(w) * $signed(a)` | `DW_mult_tc` (two's-complement multiplier) | `.../pe/mult_30`, 3 873 µm² |
| `$unsigned(w) * $unsigned(a)` | `DW_mult_uns` | `.../pe/mult_28`, 4 089 µm² |
| `case (csr_sel)` read mux | AND/OR/`oai222` tree | the critical path in `u_csr` |
| skew / de-skew shift registers | plain flops, no logic between | 48 + 192 flops |

The PE is worth a closer look. Its RTL chooses the multiplication with an `if/else`:

```
always_comb begin
  if (mode_unsigned) product = $unsigned(weight_reg) * $unsigned(act_in);
  else               product = 32'($signed(weight_reg) * $signed(act_in));
end
```

DC cannot share one multiplier between a signed and an unsigned operation written this way, so it builds **both** and a 32-bit mux to choose between them:

**Inside one PE (~14,600 µm²): two multipliers where one would do**




unpu_pe.sv selects signed or unsigned multiplication with an if/else, so DC builds BOTH multipliers and a 32-bit mux.

**Across 16 PEs the two multipliers are 127,390 µm² — 29.7 % of the whole macro.**

Possible saving (estimate, not run): one 9×9 signed multiplier on sign- or zero-extended operands covers both modes.
At ~(81/64)× the 8×8 signed multiplier's area that is ≈ 4,900 µm² per PE instead of 7,962: ≈ 49,000 µm² (~11 %) saved.

It changes rtl/unpu_pe.sv, so it would reopen the RTL freeze and need a full re-verification. Decide with the team first.

Area inside one PE

Chapter 10 returns to this, because it is the largest single area item in the macro.

# 4. Static timing in one chapter

DC does not simulate. It checks timing by **static timing analysis (STA)**: it adds up delays along every path from a launch point to a capture point, with no test vectors.

**A timing path** starts at an input port or a flop's clock pin and ends at an output port or a flop's data pin. The uNPU has four kinds:

| Kind | Example |
|---|---|
| input → register | `dma_rdata` → DMA `stage` flops |
| register → register | `weight_reg` → multiplier → adder → `psum_out` |
| register → output | DMA state flop → `dma_valid` |
| input → output (combinational) | `paddr` → CSR read mux → `prdata` |

**Setup check.** Data launched at one clock edge must arrive before the next edge, minus the flop's setup time and minus the clock uncertainty:

```
slack(setup) = required − arrival
required     = period − uncertainty − setup time   (or − output delay, at a port)
arrival      = launch time + input delay + Σ cell delays + Σ wire delays
```

Positive slack: met. Negative: the chip would fail at this frequency.

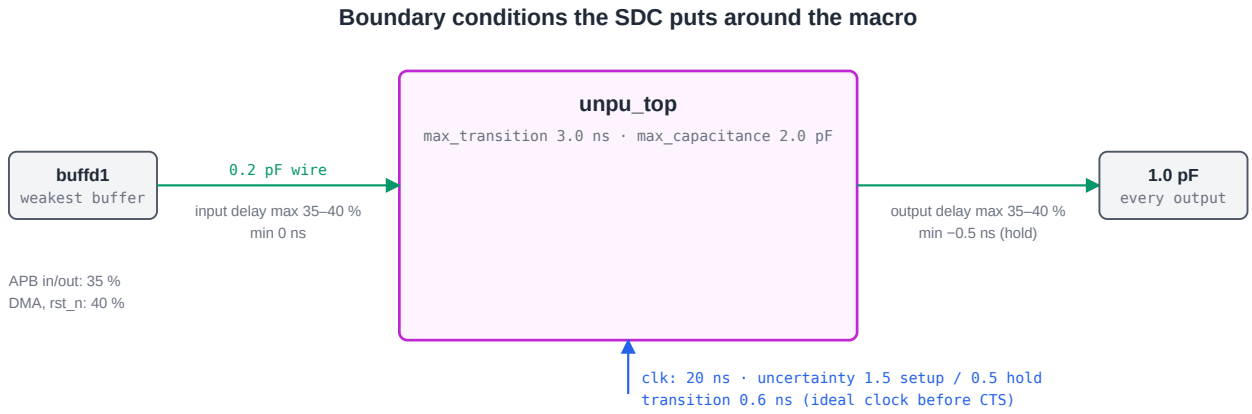
**Hold check.** New data must not arrive *too soon*, before the capturing flop has safely taken the old value. Hold is checked at the fast corner and depends on clock skew, so it is only meaningful after the clock tree exists.

**Clock uncertainty** reserves part of the period for jitter and for skew between flops. **Transition (slew)** is how long an edge takes to rise or fall; slow edges make the next cell slower, so the library limits it (`max_transition`). **Wire delay** is unknown before layout, so DC estimates each net's capacitance from its fanout using a **wire-load model** (`140000` here).

# 5. The uNPU's constraints

DC is only as good as the constraints it is given. Missing a constraint does not cause an error; the path simply goes unchecked. The file is `constraints/pm_unpu_top.sdc`.

Only two numbers are facts: the **20 ns clock** and the instructor's **1 pF output load**. Every other number is an **assumption, deliberately harsh**, until the SoC team provides real interface timing: if the design closes with these numbers, it has margin.




The boundary the SDC describes

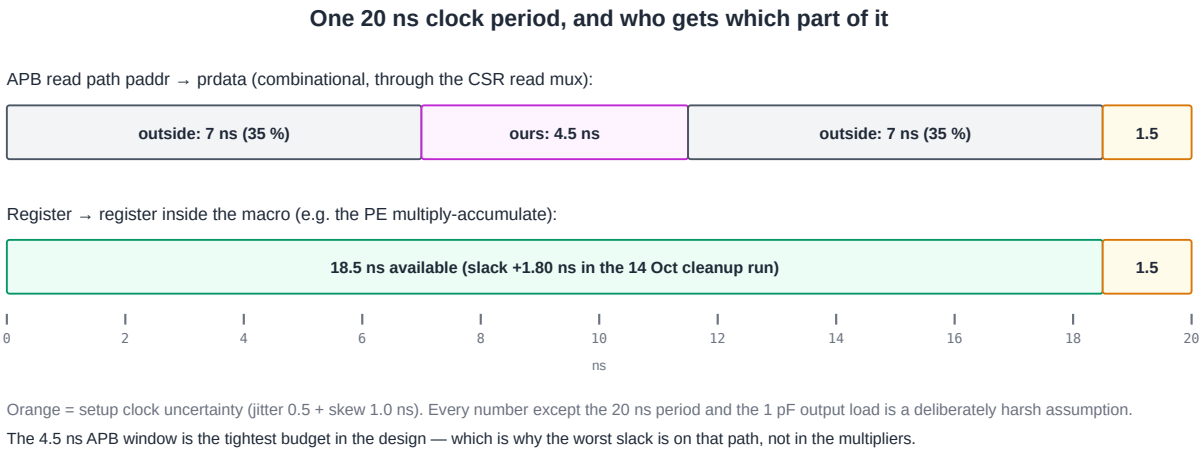
## 5.1 Clock

```
        create_clock -name clk -period 20 [get_ports clk]
        set_clock_uncertainty -setup 1.5 [get_clocks clk]   ;# jitter ~0.5 + skew ~1.0 = 7.5 %
of the period
        set_clock_uncertainty -hold  0.5 [get_clocks clk]   ;# skew only; jitter does not hurt
hold
        set_clock_transition  0.6        [get_clocks clk]   ;# slew at the flops' clock pins
(ideal clock)
```

## 5.2 How much of the period the outside world uses

```
        set_input_delay  -max  7.0  APB inputs          ;# 35 %
        set_input_delay  -max  8.0  DMA inputs, rst_n   ;# 40 %
        set_output_delay -max  7.0  prdata, pready      ;# 35 %
        set_output_delay -max  8.0  dma_* outputs       ;# 40 %
        set_input_delay  -min  0.0  all inputs          ;# may change right at the edge
        set_output_delay -min -0.5  all outputs         ;# outside needs 0.5 ns hold
```

`rst_n` is timed like any other input, with no false path: its *release* must meet recovery and removal checks even though its *assertion* is asynchronous.




Where the 20 ns go

The APB read data `prdata` is a combinational function of `paddr`, so that path must fit into what both sides leave: `20 − 7 − 7 − 1.5 = 4.5 ns`. Paths inside the macro get `20 − 1.5 = 18.5 ns`.

## 5.3 Electrical conditions at the boundary

```
        set_driving_cell -lib_cell buffd1 -pin Z  <all data inputs>   ;# weakest plain buffer
        set_load 0.2  <all data inputs>                               ;# ~1 mm of 180 nm wire
        set_load 1.0  <all outputs>                                   ;# instructor's value
        set_max_transition  3.0 [current_design]                      ;# the library's own pin
limit
        set_max_capacitance 2.0 [current_design]
```

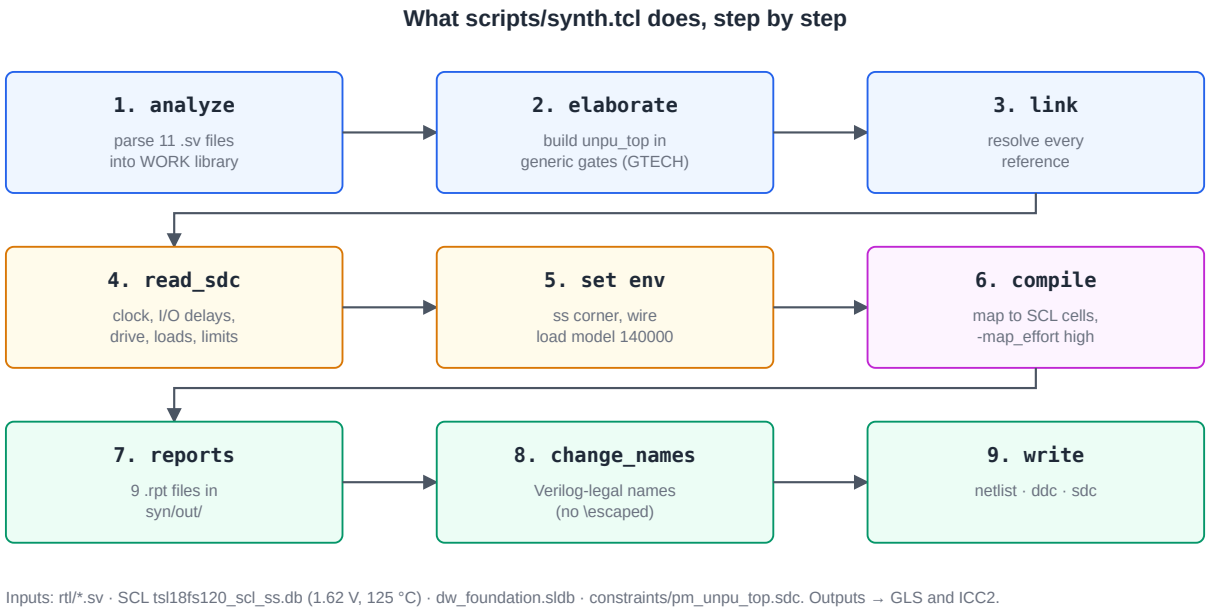
## 5.4 Operating environment (in the script)

```
        set_operating_conditions -library tsl18fs120_scl_ss tsl18fs120_scl_ss
        set_wire_load_model -name "140000" -library tsl18fs120_scl_ss
```

# 6. Running Design Compiler

On the institute server, in csh, from the repository root:

```
source ~/cad_cshrc
source ~/synopsys_cshrc
mkdir -p logs
dc_shell -f scripts/synth.tcl |& tee logs/dc_synth.log
```




What the script does

The script, condensed to the lines that do work:

```
define_design_lib WORK -path syn/out/pm_flow/WORK          ;# keep WORK files out of the
repo root
foreach ff $rtl_files { analyze -library WORK -format sverilog $ff }
elaborate unpu_top ; current_design unpu_top ; link
read_sdc constraints/pm_unpu_top.sdc
set_operating_conditions ... ; set_wire_load_model -name "140000" ...
set_max_area 0                                              ;# minimise area
set_fix_multiple_port_nets -buffer_constant -all
remove_unconnected_ports [find -hierarchy cell {"*"}]
compile -map_effort high
<nine reports>
change_names -rules verilog -hierarchy
write -hierarchy -format verilog -output syn/out/unpu_top_cn.dc.v
write -hierarchy -format ddc     -output syn/out/unpu_top_cn.ddc
write_sdc -version 2.1 syn/out/unpu_top_cn.sdc
```

Three lines exist specifically to make the netlist usable downstream:

- **`set_fix_multiple_port_nets`**: no output port is a straight feed-through of another port or tied to a constant without a buffer. Place-and-route tools dislike both.
- **`change_names -rules verilog`**: replaces names like `\g_row[0].g_col[1].pe/psum_out_reg[3]` with legal identifiers. The 14 Oct cleanup took escaped-name lines from 9 142 to 0, and `assign`

statements from 16 to 0.
- `write_sdc`: the constraints as DC applied them, with every port expanded, for ICC2.

On 15 Oct the whole run took 31.96 s of wall time.

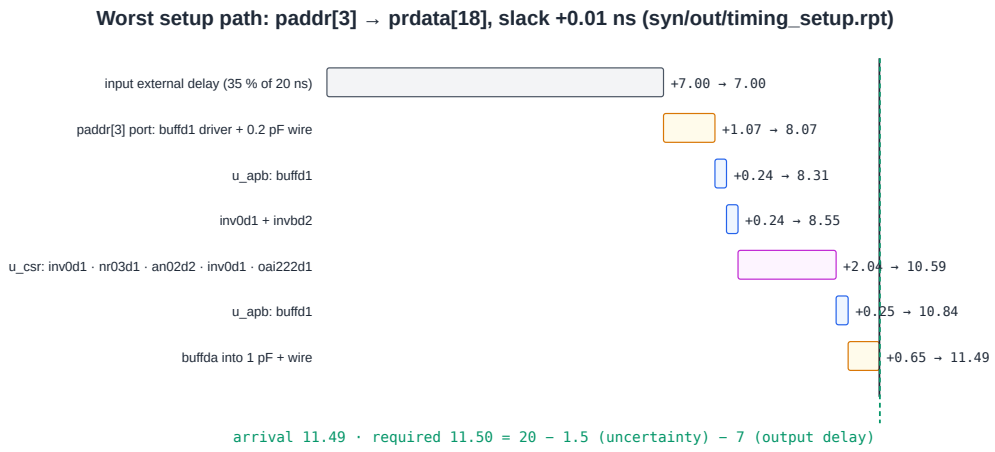
# 7. Reading the timing report

`syn/out/timing_setup.rpt` lists the ten worst setup paths. The worst one, annotated:

```
Startpoint: paddr[3] (input port clocked by clk)
Endpoint:   prdata[18] (output port clocked by clk)

Point                                   Incr       Path
clock clk (rise edge)                   0.00       0.00
input external delay                    7.00       7.00 r   <- SDC: 35 % used outside
paddr[3] (in)                           1.07       8.07 r   <- buffd1 driving 0.2 pF
u_apb/U29/Z (buffd1)                    0.24       8.31 r
U37/ZN (inv0d1)                         0.09       8.40 f
U36/ZN (invbd2)                         0.15       8.55 r
u_csr/U20/ZN (inv0d1)                   0.20       8.75 f   <- address decode ...
u_csr/U21/ZN (nr03d1)                   0.56       9.31 r
u_csr/U13/Z (an02d2)                    0.48       9.79 r
u_csr/U29/ZN (inv0d1)                   0.39      10.18 f
u_csr/U392/ZN (oai222d1)                0.41      10.59 r   <- ... and read mux
u_apb/U32/Z (buffd1)                    0.25      10.84 r
U58/Z (buffda)                          0.64      11.48 r   <- strong buffer for 1 pF
prdata[18] (out)                        0.01      11.49 r
data arrival time                                 11.49

clock clk (rise edge)                  20.00      20.00
clock uncertainty                      -1.50      18.50
output external delay                  -7.00      11.50   <- SDC: 35 % used outside
data required time                                11.50
slack (MET)                                        0.01
```




10 levels of logic: 4.49 ns of the design's 4.50 ns window. DC stops optimising once slack is positive, so +0.01 ns is not margin.
The path is APB address decode + CSR read mux. Smaller real interface budgets from the SoC team would relax it; nothing in the PE is involved.

The same path as a waterfall

How to read it:

- The `r`/`f` column shows the edge direction at each pin; inverters flip it.
- DC put a **strong `buffda`** on the output because it must drive 1 pF. On the input side, the slow `buffd1` driver into 0.2 pF costs over 1 ns before any of our logic is reached.
- **+0.01 ns is not a warning.** DC stops working on a path once it meets timing; it does not create margin it was not asked for. All ten reported paths are this same APB read path, with slack +0.01 to

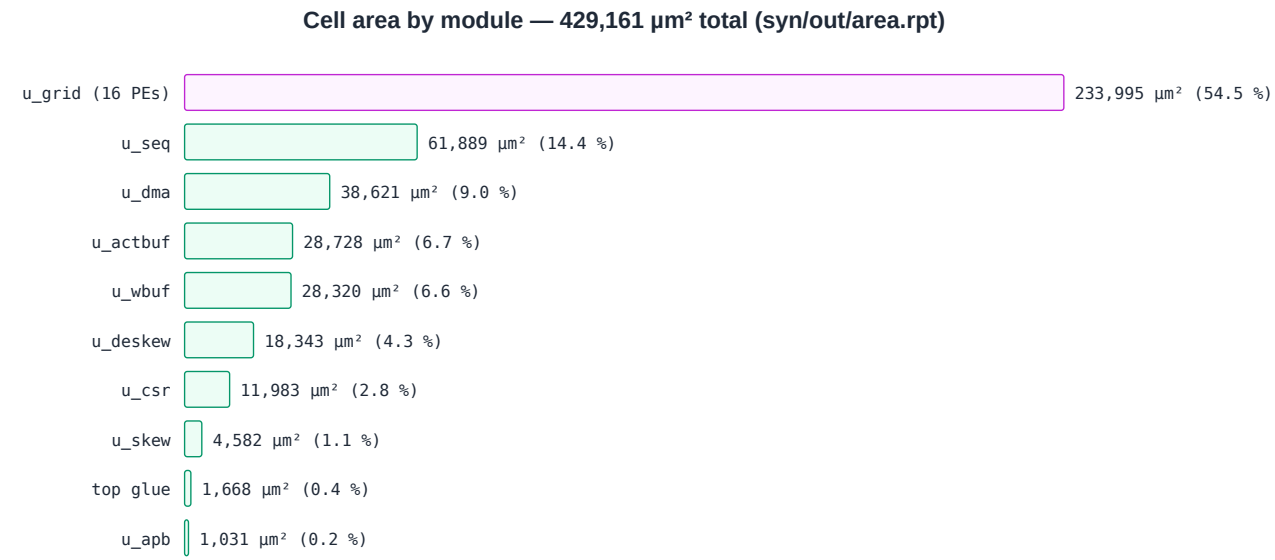
+0.09 ns.

- **The paths you don’t see.** Because the report lists only the ten worst, the register-to-register paths, which have far more room, never appear. The 14 Oct cleanup run measured them at **+1.80 ns** (the PE multiply-accumulate). `syn/extra_reports.tcl` lists them for the current netlist.

`timing_hold.rpt` shows the same for hold (worst +0.17 ns, `u_csr/start_pulse_reg` → `status_done_reg_reg`). It uses the ss library and an ideal clock, so it is only a sanity check.

# 8. Area, cells, power, rule checks

## 8.1 Area




Flops: 2,465 (noncombinational area 178,149 µm², 41.5 %). u_seq is large because it holds the 4×4×32-bit result matrix c_dst (512 flops).

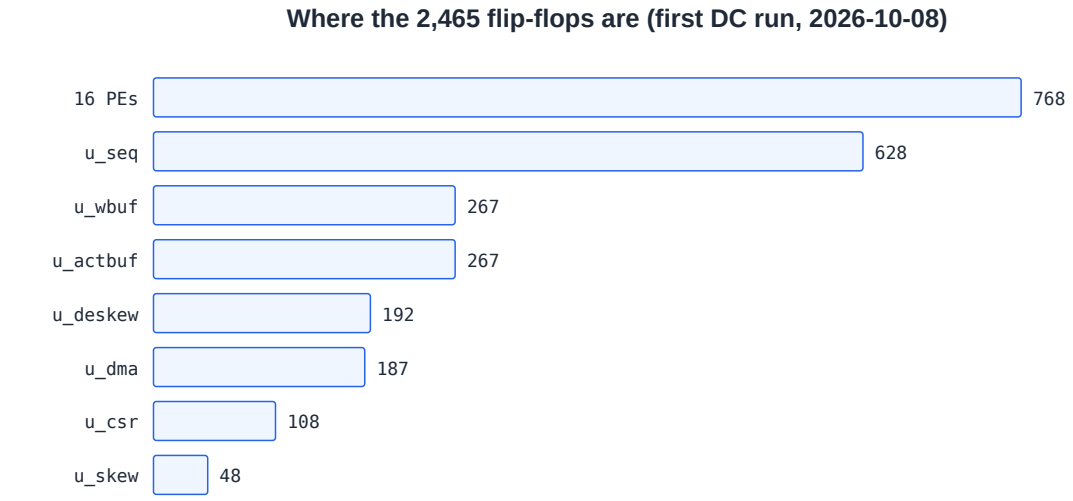
Cell area by module

| | µm² |
|---|---|
| Combinational | 251 012 |
| Noncombinational (2 465 flops) | 178 149 |
| **Cell area** | **429 161** |
| Net estimate (wire-load model) | 13 135 |
| **Total** | **442 296** (0.44 mm²) |

The 16 PEs are 54.5 % of the macro, and inside them the two multipliers per PE (chapter 3) are 127 390 µm²: **29.7 % of everything**. `unpu_seq` is second because it stores the full 4 × 4 × 32-bit result matrix (512 flops) before write-back.

These numbers feed the floorplan: at a typical 60–70 % placement utilisation, 429 161 µm² of cells needs roughly 0.61–0.72 mm² of macro, which is where the ~820 × 820 µm macro proposal comes from.

## 8.2 Flip-flops




The RTL resets asynchronously (active-low rst_n in all 28 always_ff blocks), so DC uses clear-able flops such as dfcrq1 / dfcrn1.

Where the flops are

## 8.3 Power

`report_power` gives 6.40 mW dynamic and 5.57 µW leakage, 94 % of it in the "clock network". **These are placeholders, not results.** The clock is ideal (there is no clock tree yet), and DC has no switching activity (`PWR-414`/`PWR-415`), so it guesses toggle rates. A real number needs the post-CTS netlist and activity from simulation (SAIF).

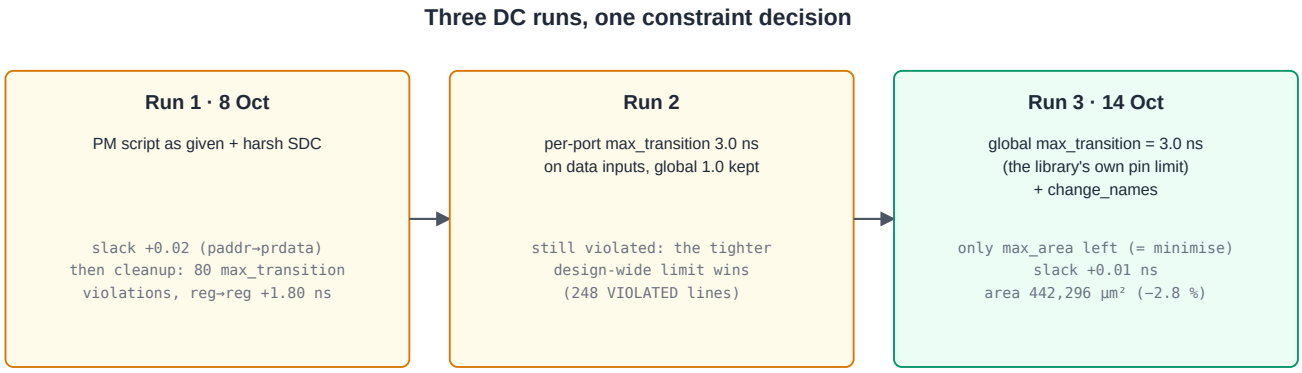
## 8.4 Rule checks

- **`check_timing`**: clean. No loops, no unconstrained endpoints, no inputs without a delay or a driver. One informational `TIM-134`: one high-fanout net (DC uses fanout 1 000 for its delay). Nets like `rst_n` and `array_en`, which reach almost every flop, are buffered by ICC2.
- **`check_design`**: no errors; 331 expected lint warnings:

| ID | Count | Meaning here |
|---|---|---|
| `LINT-28` | 111 | Unconnected ports: `paddr[31:12]`, `paddr[1:0]` (by design), unused carry pins of DesignWare adders |
| `LINT-32` | 201 | Cell pins tied to constants: the grid's top-row `psum_in` (0), constant adder bits, unused `error_code` bits |
| `LINT-33` | 19 | One constant net on several pins of one cell |

- **`report_constraint -all_violators`**: only `max_area` (required 0, actual 442 295.84). `set_max_area 0` means "minimise"; it can never be met.
- **Design rules** (`qor.rpt`): 14 543 nets, 0 max-transition and 0 max-capacitance violations.

# 9. Three runs and one constraint lesson




Why: buffd1 driving 0.2 pF already gives a 2.43 ns input edge, so a 1.0 ns limit could never be met at the ports. Driver and wire were NOT relaxed.

Run history

The first runs found **80 `max_transition` violations**. 79 of them were on data input ports, each at 2.43 ns against a 1.0 ns design-wide limit. The cause was in the constraints, not the design: a `buffd1` driving 0.2 pF *by itself* produces a 2.43 ns edge, so the 1.0 ns limit contradicted the driver assumption, and DC had no way to fix a port.

**Attempt 1:** keep 1.0 ns design-wide and set 3.0 ns on the input ports only. DC wrote the port limits into the SDC but kept reporting violations: **when a design-wide and a port-level `max_transition` both apply, the tighter one wins.**

**Decision (run 3):** use the library's own pin limit, 3.0 ns, design-wide. The 1.0 ns had been a guess with no library basis. The pessimistic driver, input wire and 1 pF output load were all kept. The 80th violation, internal net `n45` (enable of 8 `weight_reg` flops in one PE, 1.46 ns), also cleared. Area dropped 2.8 %, mostly buffers DC no longer added to chase an impossible limit.

The lesson generalises: **when DC cannot fix a violation, check the constraint before the design.**

# 10. Findings and recommendations

**1. One multiplier per PE instead of two.** The two DesignWare multipliers are 29.7 % of the macro. One 9 × 9 signed multiplier on sign- or zero-extended operands computes both modes:

```
        logic signed [8:0] w9, a9;
        assign w9 = {~mode_unsigned & weight_reg[7], weight_reg};   // sign-extend if signed,
zero-extend if not
        assign a9 = {~mode_unsigned & act_in[7],     act_in};
        assign product = 32'(w9 * a9);                             // 255 × 255 = 65 025 fits
in 18 signed bits
```

A 9 × 9 multiplier is roughly 81/64 of an 8 × 8 one, about 4 900 µm² against 7 962 today. That saves about 3 000 µm² per PE and **~49 000 µm² (~11 %) overall**, and likely shortens the PE path too. *This is an estimate; it was not synthesised.* It changes frozen RTL, so it needs the whole test suite and GLS again. It is a team decision, worth taking only if the floorplan is tight.

**2. Treat +0.01 ns as a budget question.** The worst path is the APB read mux inside a 4.5 ns window. Real APB delays from the SoC team will almost certainly be smaller than 35 % each side. If they really are that large, registering `prdata` (one APB wait state) is the design fix, at the cost of an RTL and protocol change.

**3. Report what the run currently hides.** Add `syn/extra_reports.tcl` after the main run: register-to-register, input-to-register and register-to-output paths, and high-fanout nets.

**4. Small SDC tidy-up.** The PM's `set_input_delay 0 [all_inputs]` also applies to `clk`. It is harmless in DC but may warn in PrimeTime.

**5. Don't quote DC's power or hold.** Both need a clock tree, and power needs real activity.

# 11. Sign-off checklist and hand-off

```
        grep -c "^Error" logs/dc_synth.log                                    # 0
        grep -c VIOLATED syn/out/violators.rpt                                # 1  (max_area)
        grep "slack (VIOLATED)" syn/out/timing_*.rpt | wc -l                  # 0
        grep -E "Max Trans Violations|Max Cap Violations" syn/out/qor.rpt      # 0, 0
        grep -cE '^\s*(slnht|skbrb|slbhb|slnhn|slnln|mx08d)' syn/out/unpu_top_cn.dc.v   # 0
        grep -c '\\' syn/out/unpu_top_cn.dc.v                                 # 0  (escaped
names)
        grep -c 'assign' syn/out/unpu_top_cn.dc.v                             # 0
        grep -E "Sequential Cell Count|Cell Area" syn/out/qor.rpt              # 2465, ~429161
```

Then the functional check: gate-level simulation of `unpu_top_cn.dc.v` must report 343 341 checks passed, as the RTL does (see `sim/README.md`). It did on 12 and 14 Oct.

**Hand-off to ICC2:** `unpu_top_cn.dc.v` (netlist), `unpu_top_cn.sdc` (constraints), and the macro size derived from the cell area.

**What synthesis does not prove:** real wire delays, hold, clock skew and power all wait for layout and PrimeTime. Functional equivalence is shown by GLS; a formal equivalence check (Formality is installed) has not been run.

# 12. Command reference card

### Environment (each session)

```
        source ~/cad_cshrc
        source ~/synopsys_cshrc
```

### Production synthesis (repo root)

```
        mkdir -p logs
        dc_shell -f scripts/synth.tcl |& tee logs/dc_synth.log
        dc_shell -f syn/extra_reports.tcl |& tee logs/dc_extra.log        # optional extra
timing reports
```

### Bring-up flow (one design, self-checking)

```
        setenv DESIGN unpu_top                    # or unpu_pe
        mkdir -p syn/out/$DESIGN ; cd syn/out/$DESIGN
        dc_shell -f ../../dc_run.tcl |& tee dc.log
        cd ../../.. ; bash scripts/dc_summary.sh $DESIGN
```

### Gate-level simulation of the result

```
        xrun -sv -64bit -access +r -timescale 1ns/1ps -notimingchecks -top unpu_top_tb -
xmlibdirname gls/top -l logs/gls_top.log syn/out/unpu_top_cn.dc.v
/storage/PDK_iitg/SCLPDK_V3.0_KIT/scl180/stdcell/fs120/4M1IL/verilog/vcs_sim_model/tsl18fs120_scl
.v tb/unpu_top_tb.sv
```

### Interactive look at the design

```
        dc_shell
        dc_shell> set target_library
/storage/PDK_iitg/SCLPDK_V3.0_KIT/scl180/stdcell/fs120/4M1IL/liberty/lib_flow_ss/tsl18fs120_scl_s
s.db
        dc_shell> set link_library "* $target_library dw_foundation.sldb"
        dc_shell> read_ddc syn/out/unpu_top_cn.ddc
        dc_shell> report_timing -from [all_registers -clock_pins] -to [all_registers -data_pins]
-max_paths 3
        dc_shell> report_area -hierarchy
        dc_shell> exit
```